

סיכום נושא: GROUP BY ואגרגציות - עלמה זיכל

רענון סוגי JOIN -

סוג JOIN	מה מחזיר	מתי משתמשים
INNER JOIN	רק רשומות שיש להן התאמה בשתי הטבלאות	כשמעניין אותנו רק מידע משותף רשומות בלי התאמה נעלמות
LEFT JOIN	כל הרשומות מהטבלה השמאלית + התאמות מהימנית	כשחשוב לא לאבד מידע מהטבלה הראשית אם אין התאמה נקבל NULL בצד ימין
LEFT JOIN + WHERE IS NULL	רק רשומות מהשמאלית שאין להן התאמה בימנית	למציאת מה שחסר
RIGHT JOIN	כל הרשומות מהטבלה הימנית + התאמות מהשמאלית	עדיף לא להשתמש, אלא להחליף בין סדר הטבלאות ולהשתמש ב LEFT JOIN

o RIGHT JOIN לא מוסיף יכולת חדשה, כל מה שאפשר לעשות איתו אפשר לעשות עם LEFT JOIN פשוט ע"י החלפת סדר הטבלאות. לכן, עדיף להימנע מ-RIGHT JOIN.

חוב משיעור שעבר, חשיבות ההגדרה המדויקת של השאלה ב-SQL -

ההבדל בין "לא ביים את מרילין" לבין "ביים מישהו שאינו מרילין"

במאי שביים סרטים, ובסרט היו תפקידים, והיה שם מישהו שהם שלו הוא לא מרלין מונרו – עונה על השאלה
במאי שביים שחקנית שאינה מונרו.

אבל אם רוצים את הבמאים שלא ביימו את מרלין זה לא מספיק טוב, כי מרלין מונרו שיחקה בסרט עם שחקנים
נוספים, והבמאי שביים אותה באותו סרט, ביים אנשים שהם לא מרלין מונרו.

לכן, צריך להגדיר את זה בצורה שונה-

כל הבמאים פחות הבמאים שכן ביימו את מרלין מונרו יתנו לנו את הבמאים שלא ביימו את מרלין.

כדי למצוא את הבמאים שלא ביימו את מרלין מונרו נלך בצעדים קטנים-

1. למצוא את מרלין-

```
select *  
from actors  
where first_name = 'Marilyn'  
and last_name = 'Monroe'  
;
```

2. סרטים שמרלין שיחקה בהם-

```
select movie_id  
from  
actors as marilyn  
join  
roles as r  
on
```

```

r.actor_id = marilyn.id
where
marilyn.first_name = 'Marilyn'
and marilyn.last_name = 'Monroe'
;

```

3. במאים שביימו סרטים שמרלין שיחקה בהם-

```

select DISTINCT md.director_id
from
actors as marilyn
join
roles as r
on
marilyn.id = r.actor_id
join
movies_directors as md
on
r.movie_id = md.movie_id
where
marilyn.first_name = 'Marilyn'
and
marilyn.last_name = 'Monroe'
;

```

· חשוב לשים לב-

o אנחנו שולפים רק את director_id משום שאנחנו מחפשים את רשימת הבמאים שביימו את מרילין מונרו, ולא את רשימת הסרטים.

o משתמשים ב-DISTINCT כדי שכל במאי יופיע פעם אחת בלבד, גם אם בייים כמה סרטים שבהם השתתפה מרילין מונרו.

4. למצוא במאים שלא ביימו את Marilyn Monroe-

```

select d.*
from
directors as d
left join
(select md.director_id
from
actors as marilyn
join
roles as r
on
marilyn.id = r.actor_id
join
movies_directors as md
on

```

```

r.movie_id = md.movie_id
where
marilyn.first_name = 'Marilyn'
and
marilyn.last_name = 'Monroe'
) as marilyn_directors
on
d.id = marilyn_directors.director_id
where
marilyn_directors.director_id is null
;

```

· דוגמה מרכזית לשימוש ב-LEFT JOIN + IS NULL למציאת רשומות שאין להן התאמה.

· **למה לשים לב:** לא מחפשים במאים שביימו אחרים, אלא במאים שלא הופיעו כלל ברשימת הבמאים שביימו את מרילין.

5. דוגמא לשאילתה לא נכונה-

```

select d.*
from
actors as marilyn
join
roles as r
on
marilyn.id = r.actor_id
join
movies_directors as md
on
r.movie_id = md.movie_id
right join
directors as d
on
md.director_id = d.id
where
marilyn.first_name = 'Marilyn'
and
marilyn.last_name = 'Monroe'
and
md.director_id is null
;

```

הסבר של הטעות-

· הגרסה הלא תקינה משתמשת ב-RIGHT JOIN כדי לצרף את כל הבמאים, אך לאחר מכן מפעילה בתנאי WHERE סינון על העמודות של Marilyn Monroe.

· ברגע שמסננים לפי מרילין, נשארות רק שורות שבהן נמצא קשר אמיתי בין מרילין, הסרט והבמאי. לכן באותן שורות md.director_id כבר חייב להכיל ערך קיים, ולא יכול להיות NULL.

- כתוצאה מכך התנאי האחרון `md.director_id IS NULL` סותר את כל התנאים הקודמים, ולכן השאילתה לא מחזירה אף תוצאה.

נקודות נוספות-

- לבחור טוב רק את השדות שצריך, אם לא צריך את כל השדות אין מה להביא את כולם כי זה פוגע בביצועים.
 - o לדוגמא- לא צריך את הבמאי שביים את מרלין ב20 סרטים שיחזור 20 פעמים. אם נבצע את השליפה ביחד עם הסרט כן תהיה חזרה מכיוון שמדובר על סרטים שונים ולהם רשומות שונות. לכן צריך לבחור רק את שם הבמאי ואת הID שלו כן נקבל אותו רק פעם אחת.
- לפני שכותבים את השאילתה בSQL זה נוח להגיד לעצמינו בעברית או באנגלית. צריך לשים לב שאנחנו מדברים על האובייקטים בסדר נכון.
- התייחסות לשימוש ב `IN\NOT IN` – בכל פעם שהוא מגיע לשורה של במאי הוא מחשב מי ביים את מרלין מונרו מה שגורם להרצה של הקוד להיות מאוד איטית. לכן, עדיף להשתמש בJOIN.

אגרגציה -

- מה זה Aggregations - פונקציות אגרגציה הן פונקציות שמבצעות חישוב על קבוצת שורות ומחזירות ערך מסכם אחד. במקום לקבל הרבה שורות, מקבלים תוצאה אחת שמסכמת את הנתונים.
- באמצעותן ניתן להפיק סטטיסטיקות ותובנות מתוך הנתונים.
- העוצמה של אגריגציה היא שהיא מאפשרת לקחת קבוצת שורות ולהפוך אותה לתוצאה חדשה ומסוכמת. לאחר שמתקבלת התוצאה החדשה, ניתן להמשיך לעבוד עליה, לקבץ אותה שוב, לבצע עליה חישובים נוספים ולבנות ניתוחים מורכבים בשלבים.
- פונקציות נפוצות -

SUM() - סכום ערכים

AVG () - ממוצע

MIN () - הערך הנמוך ביותר

MAX () - הערך הגבוה ביותר

STDDEV () - סטיית תקן

COUNT () - שימושים אפשריים -

o לספור את כל הרשומות בקבוצה -

COUNT(*)

o לספור רק רשומות שבהן הערך אינו NULL -

COUNT(column)

o לספור כמה ערכים שונים קיימים בקבוצה -

COUNT(DISTINCT column)

GROUP_CONCAT () - מחבר כמה ערכים ממספר שורות למחרוזת אחת.

o במקום לקבל הרבה שורות נפרדות, מקבלים שורה אחת שמכילה את כל הערכים בקבוצה.

o הפונקציה שימושית במיוחד כאשר רוצים "להציץ פנימה" אל תוך הקבוצות ולראות אילו ערכים קיימים בכל קבוצה, ולא רק לקבל מטריקות כמו COUNT או AVG.

o ברירת המחדל היא שהערכים מופרדים בפסיקים, אך ניתן לשנות את המפריד באמצעות SEPARATOR -

GROUP_CONCAT(role SEPARATOR ' | ')

o ניתן להשתמש ב-DISTINCT כדי למנוע כפילויות בתוך הרשימה -

GROUP_CONCAT(DISTINCT role)

o מיין הערכים בתוך הרשימה -

GROUP_CONCAT(role ORDER BY role)

o דוגמה כללית-

```
SELECT movie_id,  
GROUP_CONCAT(role)  
FROM roles  
GROUP BY movie_id;
```

· השאילתה מאפשרת לראות את כל התפקידים שמופיעים בכל סרט במקום לקבל שורה נפרדת לכל תפקיד.

· חשוב לשים לב: GROUP_CONCAT() מחזירה מחרוזת אחת ארוכה ולכן פחות מתאימה לקבוצות גדולות מאוד.

- GROUP BY

· משמש לחלוקת הנתונים לקבוצות לפי עמודה אחת או יותר.

o כל הערכים הזהים בעמודת הקיבוץ נכנסים לאותה קבוצה.

· לאחר החלוקה לקבוצות, ניתן להפעיל פונקציות אגרגציה על כל קבוצה בנפרד, כך אפשר לקבל מידע מפורט יותר במקום תוצאה אחת על כל הטבלה.

· סדר פעולות-

FROM - בחירת הטבלאות

WHERE - סינון רשומות לפני קיבוץ

GROUP BY - יצירת קבוצות

HAVING - סינון קבוצות לאחר קיבוץ

ORDER BY - מיון התוצאה הסופית

· אם מפעילים פונקציית אגרגציה בלי GROUP BY - החישוב יתבצע על כל הטבלה.

· אם מפעילים פונקציית אגרגציה יחד עם GROUP BY - החישוב יתבצע בנפרד עבור כל קבוצה.

· לאחר שמבצעים GROUP BY, כל קבוצה הופכת לשורה אחת. לכן כבר אי אפשר לשלוף את הרשומות הגולמיות של הקבוצה כי לכל קבוצה יש כמה שורות ולא ברור איזה ערך לבחור. **ניתן לשלוף רק את עמודות הקיבוץ או פונקציות אגרגציה על הקבוצה.**

o דוגמה לשאילתה שלא תעבוד כי לא ניתן לשלוף את העמודות הגולמיות לאחר GROUP BY

```
SELECT *  
FROM animals  
GROUP BY `Animal Group`;
```

· אפשר לקבץ לפי יותר מעמודה אחת-

שנה + ז'אנר-

```
SELECT year, genre, COUNT(*)
FROM movies
GROUP BY year, genre;
```

חשיבות ההגדרה המדויקת של המדד -

שלושת השאלות הבאות כתובות בצורה דומה אך הן שונות מאוד בתוצאה, וצריך לחשוב מה אנחנו רוצים לקבל לפני שמריצים את השאלתה - לספור באופן כללי כמה תפקידים השחקן שיחק / מספר התפקידים השונים שכל שחקן שיחק / מספר הסרטים השונים בהם כל שחקן השתתף. השאלה מה אנחנו רוצים לשאול היא לא שאלת SQL, אנחנו נצטרך להחליט מה התשובה לשאלה, זו בחירה שלנו.

מציאת מספר כל התפקידים של כל שחקן-

```
SELECT actor_id,
COUNT(*) AS roles_count
FROM roles
GROUP BY actor_id;
```

· למה השאלתה חשובה: דוגמה בסיסית ל-GROUP BY עם COUNT(*).

· חשוב לשים לב: כל שורה נספרת, גם אם אותו תפקיד או אותו סרט חוזרים.

מציאת מספר התפקידים השונים שכל שחקן שיחק ע"י שימוש ב-DISTINCT-

```
SELECT a.id AS actor_id,
a.first_name,
a.last_name,
COUNT(DISTINCT r.role) AS num_of_roles
FROM actors AS a
JOIN roles AS r
ON a.id = r.actor_id
GROUP BY a.id, a.first_name, a.last_name
ORDER BY num_of_roles DESC;
```

· חשוב לשים לב: אם אותו תפקיד חזר כמה פעמים, הוא יספר פעם אחת בלבד.

בכמה סרטים שונים כל שחקן השתתף-

```
SELECT actor_id, COUNT(DISTINCT movie_id) AS num_of_movies
FROM roles
GROUP BY actor_id;
```

· למה השאלתה חשובה: מלמד להבדיל בין ספירת שורות לבין ספירת סרטים שונים

· חשוב לשים לב: אם לשחקן היו כמה תפקידים באותו סרט, הסרט יספר פעם אחת בלבד.

· WHERE vs HAVING -

· WHERE מסנן רשומות גולמיות לפני הקיבוץ.

· HAVING מסנן קבוצות לאחר ה-GROUP BY.

דוגמא- קבוצות של חיות שלא חיות בסוואנה או יער, ורק אם יש יותר מ-2 חיות בקבוצה-

```
SELECT `Animal Group`,
COUNT(*) AS cnt
FROM animals
WHERE Habitat NOT IN ('Savanna', 'Forest')
GROUP BY `Animal Group`
HAVING COUNT(*) > 2;
```

• השאילתה מדגימה את ההבדל בין סינון שורות לפני הקיבוץ לבין סינון קבוצות לאחר הקיבוץ.

• חשוב לשים לב - WHERE - פועל על הרשומות המקוריות לפני הקיבוץ. HAVING - פועל על הקבוצות שנוצרו לאחר ה-GROUP BY.

דוגמאות מתקדמות ל-GROUP BY -

קישור לקוד של הדוגמאות

שאילתה שמקבצת את החיות לפי Animal Group, מחשבת אגרגציות שונות עבור כל קבוצה, ולאחר מכן מסננת רק קבוצות שמכילות יותר מחיה אחת-

```
SELECT
`Animal Group`,
GROUP_CONCAT(`Animal Name` ORDER BY `Animal Name`) AS names,
COUNT(*) AS animals,
COUNT(DISTINCT Habitat) AS habitats,
COUNT(DISTINCT Diet) AS diets,
COUNT(DISTINCT `Body Covering`) AS body_coverings,
COUNT(DISTINCT Movement) AS movements
FROM animals
GROUP BY `Animal Group`
HAVING COUNT(*) > 1
ORDER BY `Animal Group`;
```

• השאילתה מדגימה את ההבדל בין סינון שורות לפני הקיבוץ לבין סינון קבוצות לאחר הקיבוץ.

בכמה סרטים שונים כל תפקיד הופיע -

```
SELECT role, COUNT(DISTINCT movie_id) AS movies_count
FROM roles
GROUP BY role;
```

• למה השאילתה חשובה: מדגים קיבוץ לפי תפקיד במקום לפי שחקן.

• חשוב לשים לב: אם אותו תפקיד הופיע כמה פעמים באותו סרט, הסרט יספר פעם אחת.

שאילתה שמחשבת אגרגציות שונות עבור כל קבוצת חיות -

```
SELECT
`Animal Group`,
GROUP_CONCAT(`Animal Name` ORDER BY `Animal Name`) AS names,
COUNT(*) AS animals,
COUNT(DISTINCT Habitat) AS habitats,
COUNT(DISTINCT Diet) AS diets,
```



```

COUNT(DISTINCT `Body Covering`) AS body_coverings,
COUNT(DISTINCT Movement) AS movements
FROM animals
GROUP BY `Animal Group`
ORDER BY `Animal Group`;

```

· למה השאילתה חשובה- מדגימה שימוש בכמה פונקציות אגרגציה שונות על אותן קבוצות, כולל שימוש ב-GROUP_CONCAT כדי לראות אילו חיות נמצאות בכל קבוצה.

דוגמה לשאילתה שרצה אבל אינה שימושית - השאילתה מקבצת את הנתונים לפי Animal Group, אבל לא מציגה את עמודת הקיבוץ עצמה, ולכן קשה להבין לאיזו קבוצה שייכת כל תוצאה.

```

SELECT
COUNT(*) AS animals,
COUNT(DISTINCT Habitat) AS habitats,
COUNT(DISTINCT Diet) AS diets,
COUNT(DISTINCT `Body Covering`) AS body_coverings,
COUNT(DISTINCT Movement) AS movements
FROM animals
GROUP BY `Animal Group`
ORDER BY `Animal Group`;

```

o ברוב המקרים כדאי להוסיף ל-SELECT את העמודה שעל פיה מקבצים.

שאילתה שמקבצת לפי Habitat ומחשבת עבור כל סביבת מחיה רשימת חיות ומדדים שונים על הקבוצה-

```

SELECT
Habitat,
GROUP_CONCAT(`Animal Name` ORDER BY `Animal Name`) AS names,
COUNT(*) AS animals,
COUNT(DISTINCT `Animal Group`) AS animal_groups,
COUNT(DISTINCT Diet) AS diets,
COUNT(DISTINCT `Body Covering`) AS body_coverings,
COUNT(DISTINCT Movement) AS movements
FROM animals
GROUP BY Habitat
ORDER BY Habitat;

```

קיבוץ לפי כמה עמודות-

```

SELECT
`Animal Group`,
Habitat,
GROUP_CONCAT(`Animal Name` ORDER BY `Animal Name`) AS names,
COUNT(*) AS animals,
COUNT(DISTINCT Diet) AS diets,
COUNT(DISTINCT `Body Covering`) AS body_coverings,

```

```

COUNT(DISTINCT Movement) AS movements
FROM animals
GROUP BY `Animal Group`, Habitat
ORDER BY `Animal Group`, Habitat;

```

· חשוב לשים לב-

- o כל שילוב ייחודי של Animal Group ו-Habitat הופך לקבוצה נפרדת.
- o מומלץ להציג קודם את עמודות הקיבוץ, כדי שיהיה ברור מה כל שורה מתארת.
- o כדאי למיין לפי אותן עמודות שמקבצים לפיהן, כדי שהתוצאה תהיה קריאה.
- o משתמשים ב-` סביב שם עמודה כאשר יש בשם רווח או כאשר שם העמודה הוא מילה שמורה.

- Subqueries

- שאילתה שנכתבת בתוך שאילתה אחרת
- תוצאה של שאילתה פנימית משמשת את השאילתה החיצונית
- מתי משתמשים -
 - o כאשר צריך לבצע את הפתרון בשלבים. קודם למצוא משהו ואז להשתמש בתוצאה כדי לסנן / לחשב / להשוות.
 - o תת שאילתות עוזרות להפוך שאילתות מורכבות למסודרות (structured) וברורות יותר, בכך שהן מפרקות את הפתרון לחלקים לוגיים קטנים.

- דוגמאות

קיבוץ לפי גודל הקבוצה - השאילתה מבצעת GROUP BY על תוצאה של GROUP BY אחר.

```

SELECT
animals,
COUNT(DISTINCT `Animal Group`) AS animals_groups_of_that_size,
GROUP_CONCAT(`Animal Group` ORDER BY `Animal Group`) AS animals_groups_of_that_size_names
FROM
(
SELECT
`Animal Group`,
COUNT(*) AS animals
FROM animals
GROUP BY `Animal Group`
) AS inSql
GROUP BY animals
ORDER BY animals;

```

· חשוב לשים לב-

- o השאילתה הפנימית מחשבת כמה חיות יש בכל קבוצת חיות.
- o השאילתה החיצונית מקבצת לפי גודל הקבוצה (animals) ובודקת כמה קבוצות קיימות מכל גודל.
- o GROUP_CONCAT מאפשר לראות אילו קבוצות שייכות לכל גודל קבוצה.

שאלתה שסופרת את מס' הז'אנרים השונים-

שלב ראשון - יצירת השאלתה הפנימית לשאלתה הבאה

סופרת כמה סרטים יש בכל ז'אנר -

```
SELECT mg.genre,  
COUNT(mg.movie_id) AS number_of_movies  
FROM movies_genres AS mg  
GROUP BY mg.genre  
ORDER BY number_of_movies DESC;
```

· חשוב לשים לב: אם סרט שייך לכמה ז'אנרים, הוא יופיע בכל אחד מהם.
שלב שני-

```
SELECT COUNT(*)  
FROM  
(  
SELECT mg.genre,  
COUNT(mg.movie_id) AS number_of_movies  
FROM movies_genres AS mg  
GROUP BY mg.genre  
) AS t;
```

· למה השאלתה חשובה- מדגים ששאלתה פנימית יכולה לשמש כטבלה לשאלתה חיצונית.
· חשוב לשים לב-

o השאלתה הפנימית מחזירה שורה אחת לכל ז'אנר, ולכן COUNT(*) סופר את מספר הז'אנרים.

o במקרה הזה ניתן היה לבצע את החישוב בצורה יעילה ופשוטה יותר באמצעות:
COUNT(DISTINCT mg.genre) מבלי ליצור שאלתה פנימית.

השאלתה הבאה מציגה שימוש ב-GROUP BY לצורך מציאת זוגות סרטים דומים- [קישור לקוד של הדוגמא](#)

לאחר שמחברים את טבלת התפקידים לעצמה לפי שם התפקיד, מקבצים לפי זוג סרטים וסופרים כמה תפקידים משותפים יש להם. ככל שמספר התפקידים המשותפים גבוה יותר, כך ייתכן שהסרטים דומים יותר ומתאימים להמלצה.

יוצרים טבלה קטנה יותר בשם short_roles כדי לעבוד מהר יותר-

```
DROP TABLE IF EXISTS short_roles;  
CREATE TABLE short_roles AS  
SELECT *  
FROM roles  
WHERE movie_id < 10000;
```

השאלתה המלאה-

```
drop table if exists movies_with_common_roles;
```

```
create table
movies_with_common_roles
as
select
f.movie_id as fid
s.movie_id as sid ,
count(distinct f.role) as common_roles_num ,
group_concat(distinct f.role order by f.role) as common_roles ,
from
short_roles as f
join
short_roles as s
on
f.role = s.role
where
f.role not in ('', 'Herself', 'Himself', 'Himself - Host'
, 'Narrator', 'Host', 'Themselves', 'Herself - Presenter' ,
'Himself - Presenter', 'Host/Narrator', 'extra')
and
f.movie_id < s.movie_id
group by
f.movie_id
, s.movie_id
having
count(distinct f.role) >= 3 ;
```

הסבר שלבי עבודה-

מחברים את טבלת התפקידים לעצמה כדי לחפש זוגות סרטים שיש להם אותו תפקיד-

```
short_roles AS f
JOIN short_roles AS s
ON f.role = s.role
```

מקבצים לפי זוג סרטים -

```
GROUP BY f.movie_id, s.movie_id
```

סופרים כמה תפקידים משותפים יש לכל זוג סרטים -

```
COUNT(DISTINCT f.role) AS common_roles_num
```

מציגים את התפקידים המשותפים -

```
GROUP_CONCAT(DISTINCT f.role ORDER BY f.role) AS common_roles
```

משאירים רק זוגות שיש להם לפחות 3 תפקידים משותפים מתוך מחשבה שאם לשני סרטים יש הרבה תפקידים דומים, אולי אפשר להמליץ על אחד מהם למי שאהב את השני-

HAVING COUNT(DISTINCT f.role) >= 3

ההערות החשובות-

o f.movie_id != s.movie_id - מונע מצב שבו סרט מושווה לעצמו. כלומר לא נקבל זוג כמו סרט A עם סרט A.

o f.movie_id < s.movie_id - משתמשים בזה אם רוצים למנוע כפילויות סימטריות. כלומר שלא נקבל גם A-B וגם B-A.

o הרשימה של התפקידים ב-NOT IN - נועדה להסיר תפקידים לא אינפורמטיביים, כמו Himself, Host, Narrator, כי הם לא באמת מעידים על דמיון עלילתי בין סרטים.

צפייה בצירופי התפקידים הנפוצים ביותר בין סרטים -

```
SELECT
common_roles,
COUNT(*) AS matches
FROM movies_with_common_roles
GROUP BY common_roles
ORDER BY matches DESC;
```

· למה השאילתה חשובה - מאפשרת לבדוק אילו שילובי תפקידים יוצרים הכי הרבה התאמות בין סרטים, וכך להעריך האם התפקידים שבהם משתמשים אכן אינפורמטיביים למערכת ההמלצות.

· חשוב לשים לב -

o אם תפקיד מסוים מופיע בהרבה מאוד התאמות, ייתכן שהוא כללי מדי ולא באמת עוזר להבדיל בין סרטים.

o באמצעות בדיקה ידנית של התוצאות ניתן לזהות תפקידים שכדאי להסיר, כמו Himself, Narrator

שאילתה שמציגה התפלגות של מספר הופעות לכל תפקיד (השאילתה סופרת לכל תפקיד כמה פעמים הוא מופיע, ואז בודקת כמה תפקידים שונים הופיעו בכל כמות הופעות) -

```
SELECT number_of_movies_per_role,
COUNT(*) AS roles
FROM
(
SELECT r.role,
COUNT(*) AS number_of_movies_per_role
FROM roles AS r
GROUP BY r.role
) AS insql
```

GROUP BY number_of_movies_per_role

ORDER BY number_of_movies_per_role;

- דוגמה ל-Subquery שמייצר טבלה זמנית ועליה עושים קיבוץ נוסף.
 - חשוב לשים לב: קודם סופרים הופעות לכל תפקיד, ואז בודקים כמה תפקידים יש בכל כמות.
- התפלגות של שחקנים לפי מספר התפקידים שלהם (השאלתה סופרת כמה תפקידים יש לכל שחקן, ואז בודקת כמה שחקנים קיימים בכל כמות תפקידים).

```
SELECT roles_per_actor,
COUNT(*) AS actors
FROM
(
    SELECT actor_id,
    COUNT(*) AS roles_per_actor
    FROM roles AS r
    GROUP BY actor_id
) AS t
GROUP BY roles_per_actor
ORDER BY roles_per_actor;
```

- התפלגות במאים לפי מספר הסרטים שביימו - קישור לקוד של הדוגמאות
- שלב ראשון** - השאלתה מקבצת את הנתונים לפי director_id וסופרת כמה סרטים משויכים לכל במאי.

```
SELECT
director_id,
COUNT(*) AS movies_per_director
FROM movies_directors
GROUP BY director_id
ORDER BY movies_per_director DESC;
```

- למה השאלתה חשובה- מדגימה שימוש בסיסי ב-GROUP BY לצורך ספירת כמות רשומות לכל ישות.
- חשוב לשים לב- כדי שיהיה ברור לאיזה במאי כל שורה מתייחסת director_id מופיע ראשון ב-SELECT

שלב שני- נשתמש בשאלתה הראשונה כדי לחשב את ההתפלגות

```
SELECT
movies_per_director,
COUNT(*) AS directors
FROM
(
    SELECT
    director_id,
    COUNT(*) AS movies_per_director
    FROM movies_directors
    GROUP BY director_id
```

```
) AS inSql
GROUP BY movies_per_director
ORDER BY movies_per_director;
```

· למה השאילתה חשובה - מדגימה שימוש ב-Subquery כדי לבצע קיבוץ על תוצאה שכבר עברה קיבוץ.
· חשוב לשים לב-

o השאילתה הפנימית יוצרת טבלה שבה כל שורה היא במאי ומספר הסרטים שביים.
o השאילתה החיצונית מקבצת לפי מספר הסרטים, וכך מקבלים התפלגות: כמה במאים ביימו סרט אחד, כמה ביימו שני סרטים וכן הלאה.

התפלגות במאים לפי מספר הסרטים שביימו, כולל במאים שלא ביימו אף סרט -

```
SELECT
movies_per_director,
COUNT(*) AS directors
FROM
(
SELECT
d.id AS director_id,
COUNT(DISTINCT md.movie_id) AS movies_per_director
FROM directors AS d
LEFT JOIN movies_directors AS md
ON d.id = md.director_id
GROUP BY d.id
) AS inSql
GROUP BY movies_per_director
ORDER BY movies_per_director;
```

· למה השאילתה חשובה- מדגימה איך שילוב של LEFT JOIN עם GROUP BY מאפשר לכלול גם רשומות שאין להן התאמה בטבלה השנייה.

· חשוב לשים לב-

o אם היינו מתחילים רק מטבלת movies_directors, היינו מקבלים רק במאים שביימו לפחות סרט אחד.
o השימוש ב-LEFT JOIN שומר גם במאים שלא מופיעים בטבלת movies_directors.
o במקרה כזה COUNT(DISTINCT md.movie_id) יחזיר 0 עבור במאים שלא ביימו אף סרט.

נקודות כלליות -

תפקיד החלוץ- להיות הראשון שמבצע את המטלה כדי לבדוק האם יש נקודות לא ברורות במטלה. בנוסף, המטלה שלו תהיה המטלה לדוגמא שתפורסם לאחר שנקבל פידבק. החלוץ מקבל נקודות בונוס עבור עבודתו.

נוכחות - הנוכחות בשיעור היא חובה. כפי שעידן אמר בשיעור, אנחנו סטודנטים בוגרים לכן אם נבצר מאיתנו להגיע לשיעור מסיבה מספיק חשובה זה יתקבל, אבל הנוכחות בשיעור היא חובה.

המלצה של עידן – לפני השיעור לעבור על [הדוגמאות מהgithub](#), להריץ אותן. ככה נוכל להכיר את החומר בצורה טובה יותר, ולהגיע לשיעור כשאנחנו כבר יודעים מה אנחנו פחות מבינים.

טיפול ב-Timeout בgithub יש הסבר איך להתמודד במצב שהשאלות כבדות ונמשכות יותר מ 30 שניות וצריך להוריד את המגבלה של עצירת השאלתה. נמצא תחת תיקיית knowledge base, תחת כותרת [Technical problems](#).

מטלות - ליד כל מטלה כתוב כמה נקודות היא, רוב המטלות הן לא חובה. אם מגישים מטלות שהן לא חובה – זה לא נבדק ידנית, לכן בגדול רוב הציונים יהיו 100 ואז המשקל של מטלות החובה מהציון הסופי ירד.

מטלת סיום – בניית מערכות המלצה על סרטים.

- יש תיאור של פרויקט בgithub תחת תיקיית exams and assignment [בקובץ project 2026](#)
- כרגע עושים רק את המטלה הראשונה – לדרג 200 סרטים שאנחנו מכירים כדי שיהיה זיהוי. של הטעם שלנו. הדירוג לא חייב להיות רק של סרטים שאנחנו אוהבים, אפשר גם לדרג סרטים שאנחנו שונאים.
- יש תבנית שאפשר להשתמש בה שמצמצמת את 300,000 למספר קטן הרבה יותר של סרטים שסביר להניח שיהיו מוכרים יותר.
- הנחיות לביצוע המטלה -
 - o המטלה מתבצעת בזוגות.
 - o כל אחד מבני הזוג מדרג 200 סרטים (אפשר לעבור על הטמפלייט הקיים, להשאיר סרטים שאתה מכיר, למחוק סרטים שלא מכירים. עבור הסרטים שאנחנו מכירים לתת דירוג והסבר).
 - o עושים insert לטבלה (יש קוד לדוגמא בתיאור של הפרויקט), מומלץ להכניס שורה שורה כדי לראות שאין תקלה, ואם תהיה תקלה שנדע לאתר אותה במהירות.
 - o יוצרים PR (PR זו בקשה כדי להעלות את הקובץ דירוג שלך לאתר של הקורס, [לינק למדריך שימוש בPR](#)) ולאחר מכן בן הזוג השני צריך להריץ את הטבלה שלך כדי לבדוק שאכן הכל בסדר – שיוצאים 200 סרטים, והוא גם מוודא שהנימוקים של הדירוג אכן הגיוניים. את המשוב שלו ניתן להעביר ישירות בPR.
 - o לאחר שכל הכיתה תגיש את הPR והם יבדקו ע"י המתרגל, נקבל טבלה עם כל הדירוגים של כולנו כדי שנוכל להשתמש בזה בעתיד